
N L P P O R T F O L I O P R O J E C T

Document Similarity Using RAKE Keyword Extraction

Ranking Presidential Inaugural Speeches by Keyword-Based Similarity Metrics

Tools	Python, NLTK, RAKE, Pandas, NumPy
Domain	Natural Language Processing
Dataset	NLTK Inaugural Corpus (59 speeches)
Techniques	RAKE, TF-IDF concepts, Similarity Scoring

Executive Summary

This project implements a RAKE-based (Rapid Automatic Keyword Extraction) similarity metric to quantify the thematic overlap between U.S. presidential inaugural addresses. Using the NLTK Inaugural Corpus spanning 59 speeches from George Washington (1789) to Joe Biden (2021), the system extracts weighted keyphrases from each speech, identifies shared keywords between any two speeches, and computes a composite similarity score. The result is a fully automated ranking system that reveals which historical speeches share the closest rhetorical and thematic DNA.

The project demonstrates proficiency in natural language processing, information retrieval, and data analysis — core skills for any data science or NLP engineering role.

Problem Statement

Given a corpus of 59 presidential inaugural speeches, the goal is to build a system that can answer: "Which historical inaugural addresses are most thematically similar to a given speech?" This requires three capabilities:

1. Keyword Extraction — Automatically identify the most important phrases in each speech using RAKE.
2. Pairwise Similarity — Quantify how similar any two speeches are based on their shared keywords.
3. Corpus-Wide Ranking — Rank all speeches by similarity to a query speech.

Technical Methodology

Understanding RAKE

RAKE (Rapid Automatic Keyword Extraction) is a domain-independent algorithm that extracts keywords by analyzing word frequency and co-occurrence patterns. Unlike supervised methods, RAKE requires no training data — it works purely from the statistical properties of the text itself.

The algorithm operates in four steps: it splits text into candidate phrases using stopwords and punctuation as delimiters, builds a word co-occurrence matrix, computes a score for each word (in this case, the degree-to-frequency ratio), and then scores each phrase as the sum of its constituent word scores.

Key Insight: The degree-to-frequency ratio metric favors words that co-occur with many different words (high degree) relative to how often they appear (frequency). This surfaces phrases that are both distinctive and contextually rich — exactly what we want for measuring thematic similarity.

The Similarity Pipeline

The similarity computation follows a three-stage pipeline, each implemented as a modular Python function:

Stage	Function	Description
1. Extract	Corpus2Keywords()	Applies RAKE to a single speech, returns a DataFrame of keywords and their scores
2. Compare	KeywordSim()	Merges two keyword DataFrames on matching keywords, averages paired scores, and sums them into a single similarity metric
3. Rank	RankSpeeches()	Iterates over all 59 speeches, computes similarity against a query speech, and returns a sorted ranking

Implementation

Task 1: Keyword Extraction

The Corpus2Keywords function retrieves raw text from the NLTK inaugural corpus, applies RAKE with the degree-to-frequency ratio metric, and returns a deduplicated DataFrame of keywords sorted by score:

```
def Corpus2Keywords(fid='2009-Obama.txt') -> pd.DataFrame:
    text = nltk.corpus.inaugural.raw(fid)

    r = Rake(ranking_metric=Metric.DEGREE_TO_FREQUENCY_RATIO,
            language='english',
            min_length=1, max_length=3)
    r.extract_keywords_from_text(text)

    scored = r.get_ranked_phrases_with_scores()
    df = pd.DataFrame(scored, columns=['score', 'keyword'])
    df = df.drop_duplicates(subset='score', keep='first')
    df = df.set_index('keyword')
    return df
```

Applied to Obama's 2009 inaugural address, this extracts keyphrases like *"stale political arguments"* (score: 9.0), *"use energy strengthen"* (score: 8.5), and *"would rather cut"* (score: 8.33). These high-scoring phrases capture the speech's core policy themes.

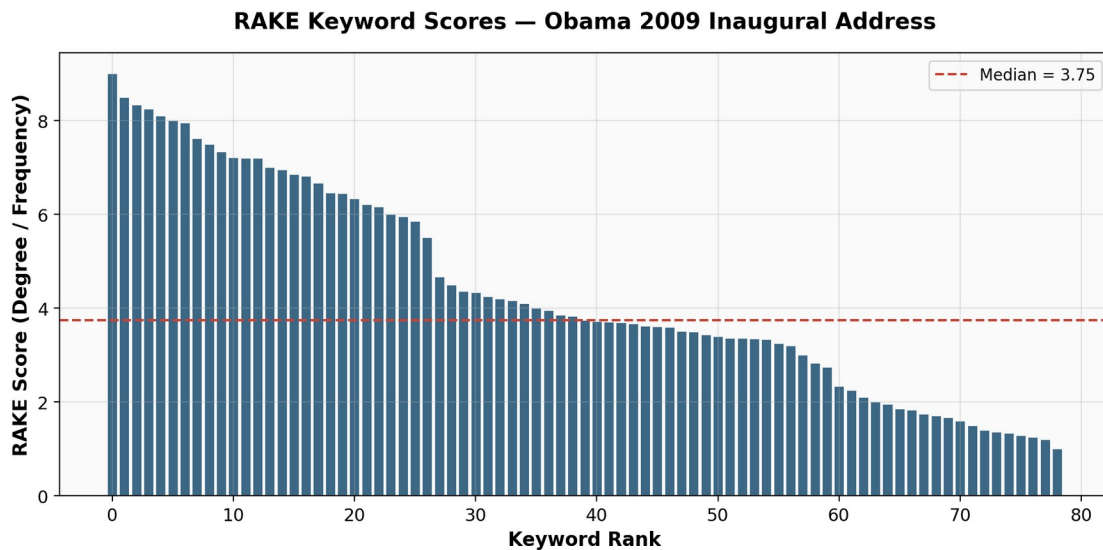


Figure 1: RAKE keyword score distribution for Obama's 2009 inaugural address. The long tail shows most phrases score low, while a few highly co-occurring phrases dominate.

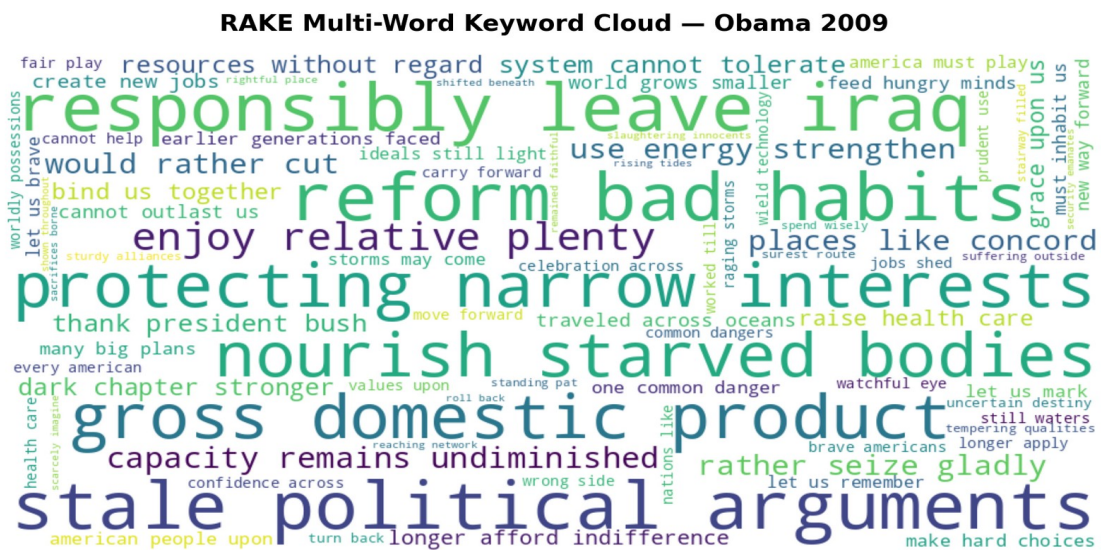


Figure 2: Word cloud of multi-word RAKE keyphrases from Obama 2009. Larger text indicates higher RAKE scores.

Task 2: Pairwise Similarity

The KeywordSim function computes similarity between two speeches by finding their shared keywords through an inner merge on the keyword index, averaging each pair of RAKE scores, and summing the results:

```
def KeywordSim(fid1='2009-Obama.txt', fid2='2013-Obama.txt') -> float:
    df1 = Corpus2Keywords(fid1)
    df2 = Corpus2Keywords(fid2)
    SimScore = df1.merge(df2, on='keyword').mean(axis=1).sum()
    return SimScore
```

This design captures two important properties: speeches that share more keywords score higher (breadth), and speeches whose shared keywords have higher RAKE scores also score higher (depth). The metric naturally handles speeches of different lengths since only matching keywords contribute.

Speech A	Speech B	Similarity Score
Obama 2009	Obama 2013	13.10
Obama 2009	Biden 2021	9.33
Obama 2009	Trump 2017	7.21

Table 1: Sample pairwise similarity scores. Obama's two speeches are most similar to each other, as expected.

Task 3: Corpus-Wide Ranking

The RankSpeeches function scales the pairwise comparison across all 59 speeches, producing a full similarity ranking for any query speech:

```
def RankSpeeches(qfid='2009-Obama.txt', FIDs=FIDs) -> pd.DataFrame:
    sims = [KeywordSim(qfid, fid) for fid in FIDs]
    df = pd.DataFrame({'Similarity': sims},
                      index=pd.Index(FIDs, name='fid'))
    df = df.sort_values('Similarity', ascending=False)
    return df
```

When queried with Biden's 2021 inaugural address, the system reveals which historical speeches share the most thematic overlap:

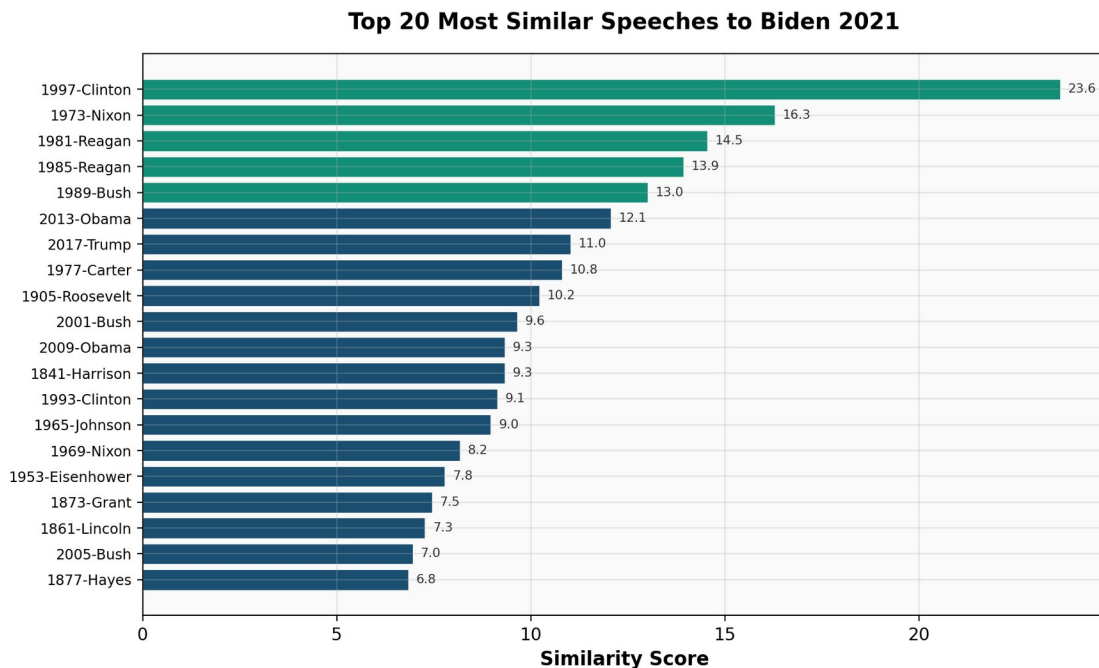


Figure 3: Top 20 speeches most similar to Biden 2021. The green bars highlight the top 5 matches.

Analysis & Insights

Cross-Presidential Similarity Patterns

The pairwise similarity heatmap for the 10 most recent inaugurals reveals interesting patterns in presidential rhetoric. Speeches from the same party or era tend to cluster, while stylistic outliers stand apart:

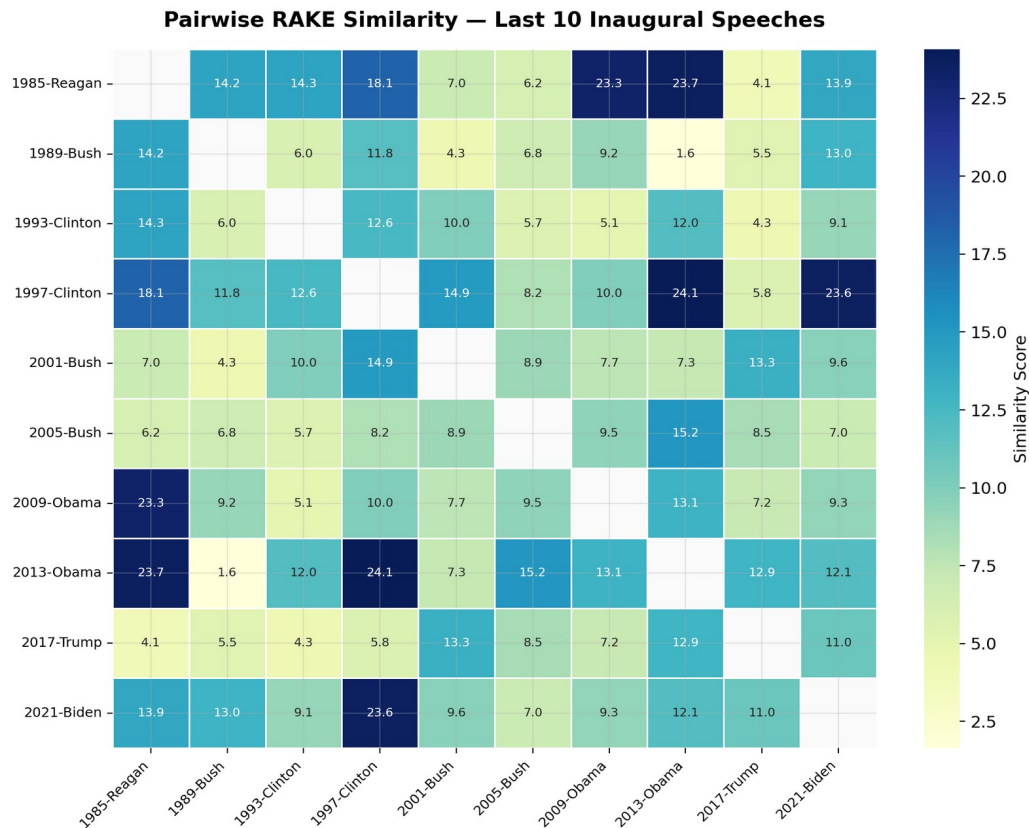


Figure 4: Pairwise RAKE similarity heatmap for the 10 most recent inaugural speeches. Diagonal values (self-similarity) are masked.

The Length Bias Problem

An important observation is the correlation between speech length and similarity scores. Longer speeches contain more keywords and therefore have more opportunities for matches.

Harrison's 1841 address (over 9,000 words — the longest in history) is similar to nearly every other speech, while Washington's brief 1793 second inaugural (just 147 words) is similar to almost none:

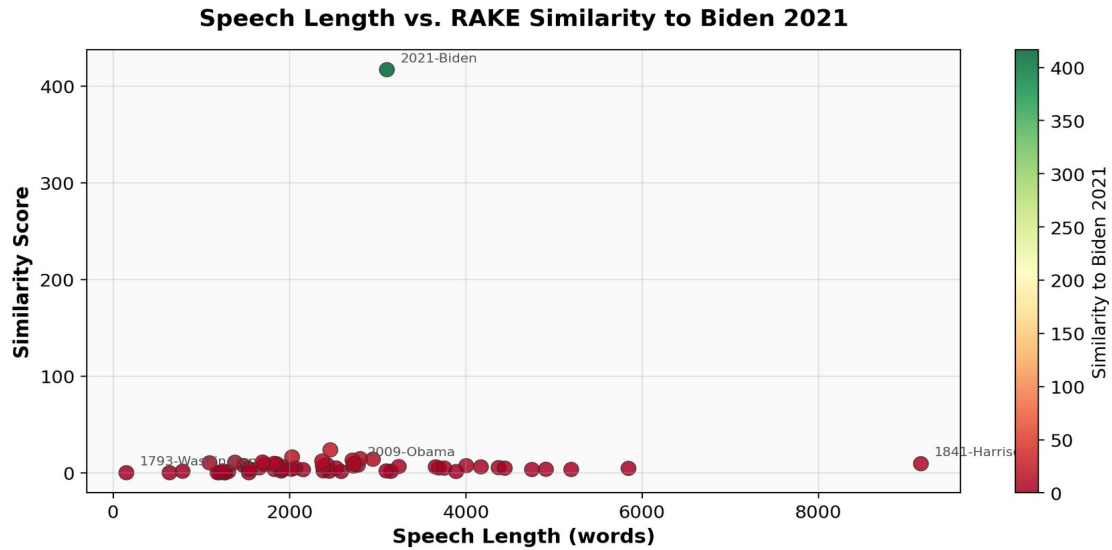


Figure 5: Speech length versus similarity score to Biden 2021. The positive correlation reveals RAKE's length bias.

Potential Improvement: Normalizing similarity scores by the geometric mean of speech lengths (or keyword counts) would control for this bias. Alternatively, TF-IDF weighting or cosine similarity over keyword vectors could produce length-invariant metrics.

Comparing Three Presidents

Examining the top RAKE keywords side-by-side reveals each president's rhetorical priorities. Obama's 2009 address emphasizes economic recovery and international engagement, Trump's 2017 speech focuses on American renewal and trade, while Biden's 2021 address centers on unity and democratic resilience:

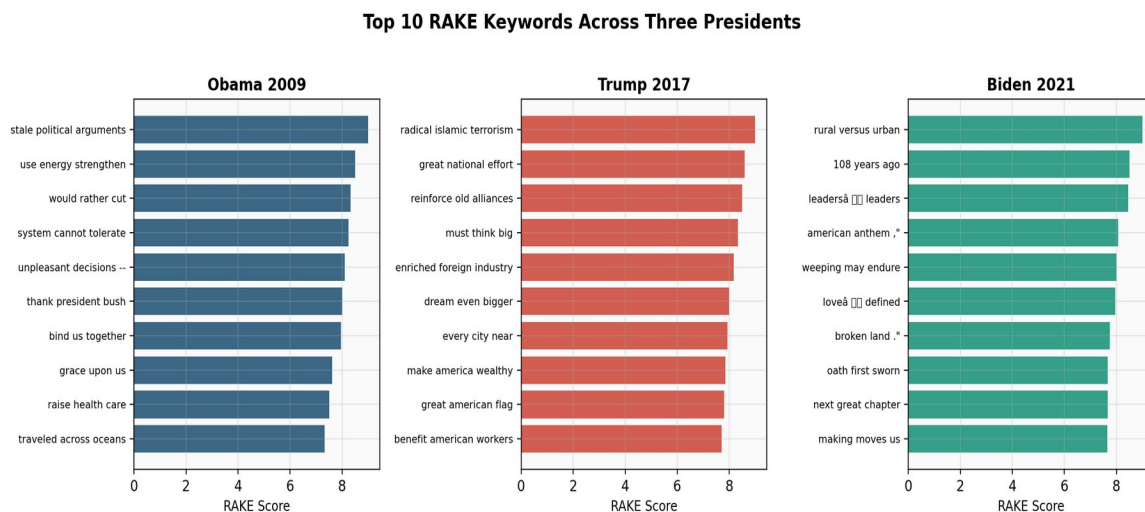


Figure 6: Top 10 RAKE keywords for Obama 2009, Trump 2017, and Biden 2021 inaugural addresses.

Technical Skills Demonstrated

Skill Area	Details
NLP & Text Processing	RAKE keyword extraction, stopwords handling, phrase scoring, co-occurrence analysis
Information Retrieval	Document similarity metrics, keyword-based ranking, inner-join matching
Python & Libraries	NLTK, rake_nltk, Pandas, NumPy, Plotly, Matplotlib, Seaborn
Data Manipulation	DataFrame merging, aggregation, multi-index operations, vectorized scoring
Data Visualization	Distribution plots, heatmaps, scatter plots, word clouds, comparative bar charts
Software Design	Modular function architecture, docstrings, type hints, unit-testable code
Critical Analysis	Identified length bias, proposed normalization solutions, evaluated metric limitations

Extensions & Future Work

While the RAKE-based similarity metric provides a strong baseline, several enhancements could improve its accuracy and applicability:

- **TF-IDF Integration:** Replace raw RAKE scores with TF-IDF weighted vectors to penalize corpus-wide common phrases and reward speech-specific terminology.
- **Cosine Similarity:** Represent each speech as a keyword vector and use cosine similarity for a length-normalized metric.
- **Semantic Embeddings:** Use sentence transformers (e.g., SBERT) to capture semantic similarity beyond exact keyword matches.
- **Topic Modeling:** Apply LDA or BERTopic to discover latent themes and compare speeches at the topic level.
- **Interactive Dashboard:** Build a Streamlit or Plotly Dash application that lets users explore speech similarities interactively.

Conclusion

This project demonstrates how unsupervised keyword extraction can be leveraged to build a practical document similarity system. By combining RAKE's statistical phrase scoring with Pandas' merge-and-aggregate operations, we created a pipeline that scales across an entire corpus while remaining interpretable — every similarity score can be traced back to specific shared keyphrases.

The analysis also revealed important limitations (length bias) and natural extensions (TF-IDF, embeddings), showing not just technical skill but the critical thinking needed to evaluate and improve NLP systems in practice.